A PROJECT REPORT
ON
FOUNDATION COURSE
(TINKERING & ELEMENTS DESIGN)

PLACEMENT ELIGIBILITY CHECKER

Submitted in partial fulfillment of the requirement for the award of Degree of
Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Nandani Kumari (24EARCS100)
Komal Agarwal (24EARCS077)
Muskan Singh (24EARCS099)
Kashish (24EARCS073)
Nency Jain (24EARCS102)

Session: 2025-26

Department of Computer Science & Engineering
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

CERTIFICATE OF APPROVAL

The Project Report entitled "**PLACEMENT ELIGIBILITY CHECKER**" submitted by "Nandani Kumari (24EARCS100), Komal Agarwal (24EARCS077), Muskan Singh (24EARCS099), Kashish (24EARCS073), Nency Jain (24EARCS102)" has been examined by us and is hereby approved for carrying out the project leading to the award of degree "**Bachelor of Technology in Computer Science & Engineering**". By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose.

Mentor
Er. Ram Babu Buri
(Assistant Professor)

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the work embodies in this Project entitled "**PLACEMENT ELIGIBILITY CHECKER**" being submitted by "**Nandani Kumari (24EARCS100), Komal Agarwal (24EARCS077), Muskan Singh (24EARCS099), Kashish (24EARCS073), And Nency Jain (24EARCS102)**" in partial fulfillment of the requirement for the award of a "**Bachelor of Technology in Computer Science & Engineering**" to Rajasthan Technical University, Kota (Raj.) during the academic year 2025-26 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the "**Department of Computer Science & Engineering**", **Arya College of Engineering & Information Technology, Jaipur**.

Mentor
Aarti Sharma
(Professor)

HOD
Dr. Akhil Panday
(Professor)

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Computer Science & Engineering

DECLARATION

We "**Nandani Kumari, Komal Agarwal, Muskan Singh, Kashish And Nency Jain**", students of Bachelor **of Technology** in **Computer Science & Engineering, session 2025-26, Arya College of Engineering & Information Technology, Jaipur**, hereby declare that the work presented in this Project entitled "**Placement Eligibility Checker**" is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Nandani Kumari (24EARCS100)
Komal Agarwal (24EARCS077)
Muskan Singh (24EARCS099)
Kashish (24EARCS073)
Nency Jain (24EARCS102)

Department of Computer Science & Engineering

ACKNOWLEDGEMENT

I would like to express my special thanks to my Project Guide **Er. Ram Babu Buri** as well and our HOD **Dr. Akhil Pandey** who gave me the golden opportunity to do this wonderful project, which also helped me in doing a lot of research and I came to know about so many new things. I am really thankful to them. Secondly I would also like to thank my parents and friends who helped me a lot in finalizing this project within the limited time frame. I am overwhelmed in all humbleness and gratefulness to acknowledge my depth to all those who have helped me to put these ideas, well above the level of simplicity and into something concrete. Any attempt at any level can't be satisfactorily completed without the support and guidance of my parents and friends. I would like to thank my parents who helped me a lot in gathering different information, collecting and guiding me from time to time in making this project, despite their busy schedules, they gave me different ideas in making this project unique.

Thanking you

TABLE OF CONTENTS

Chapter 1: Objective & Scope 3
1.1 Introduction 3
1.2 Objective 3
1.3 Scope of Placement Eligibility Checker 4

Chapter 2: Theoretical Background 6
2.1 Fundamental Concept 6
2.2 Placement Eligibility Criteria 6
2.3 Benefits of the System 7

Chapter 3: Problem Definition & System Analysis 8
3.1 Existing System & Problem Definition 8
3.2 Proposed System 8
3.3 System Analysis 9
3.4 Functional Requirements 9
3.5 Non-Functional Requirements 10

Chapter 4: System Planning 11
4.1 Introduction to System Planning 11
4.2 Step-Wise Activities 11
4.3 PERT Chart Flow (Sequence) 11

Chapter 5: Methodology Adopted and Details of Hardware and Software Used 14
5.1 Methodology Adopted 14
5.2 Hardware Requirements 14
5.3 Software Requirements 15
5.4 Technologies Used 15

Chapter 6: Detailed Life Cycle 16
6.1 System Development Phases 16
6.2 System Design and Implementation 16
6.3 Testing, Development and Maintenance 17

Chapter 7: ERD & DFD 19
7.1 Entity Relationship Diagram 19
7.2 Data Flow Diagram 20
7.3 System Architecture Diagram 21

Chapter 8: Process, Algorithm, and Database Structure.....22
8.1 System Process.....22
8.2 Algorithm.....22
8.3 Database Design.....24
Chapter 9: Input and Output Screen.....26
9.1 User Interface Overview.....26
9.2 Student Details Input Screen.....26
9.3 Eligibility Result Screen.....26
Chapter 10: Coding (Java Implementation).....28
10.1 Java Program Structure.....28
10.2 Eligibility Checking Module.....28
10.3 Database Connectivity Using JDBC.....31
Chapter 11: Conclusion.....33
Chapter 12: Future Enhancement.....34
References.....35

Ch- 1
Objective & Scope

1.1 Introduction

The Placement Eligibility Checker is a Java-based application developed to check whether students are eligible for campus placement according to predefined company requirements. In engineering colleges, many companies visit the campus every year to recruit students for different job roles. Each company publishes its own eligibility criteria, which commonly include a minimum academic percentage, a minimum CGPA, and a limit on the number of active backlogs.

Campus placement is one of the most important activities of a technical institution. It connects final-year students with employers and gives them an early start to their careers. Before a student can sit for a company's written test or interview, the placement department must confirm that the student satisfies the conditions set by that company. This confirmation step is called eligibility checking.

Checking the eligibility of every student manually can be a time-consuming and difficult task, especially when a large number of students participate in placement activities. Marks must be read from several records, percentages and CGPA values must be compared with the company's limits, and backlog details must be verified one student at a time. Manual checking may also lead to mistakes while calculating marks or verifying student details, and the same rule may be applied differently by different people.

The Placement Eligibility Checker helps solve this problem by automating the eligibility checking process. A student, or a member of the placement department on the student's behalf, enters the academic details, and the system compares the information with the specified placement criteria. It then displays whether the student is eligible or not eligible, together with the reason for the decision.

This project uses Java programming to perform the eligibility checking process. The core application described in this report is a console-based program built with object-oriented classes. If database connectivity is included, student details and results can also be stored and managed systematically; in this report, database support through JDBC is presented as an optional extension. The system helps save time, reduce manual effort, and make the placement process more organised.

1.2 Objective

The main objective of the Placement Eligibility Checker is to make the process of checking student eligibility simple, fast, and efficient. The system is designed so that a student record can be checked against the placement criteria in a few seconds, and the outcome is explained in plain language.

The major objectives of this system are:

- To check whether a student meets the required placement criteria, namely minimum percentage, minimum CGPA, and the permitted number of active backlogs.
- To reduce manual work and save time during the placement process, both for students and for the placement department.

- To provide clear results showing whether a student is eligible or not eligible for placement, along with the specific reason when the student is not eligible.
- To maintain student academic details in an organised manner, where database storage is implemented.
- To improve the accuracy, consistency, and efficiency of the eligibility checking process by applying the same rules to every student.
- To build a simple, well-structured Java application that can later be extended with company-wise criteria and a graphical interface.

1.3 Scope of Placement Eligibility Checker

The scope of the Placement Eligibility Checker describes the features and functions provided by the application. The system is designed to check student eligibility for campus placement based on the criteria defined in the program.

The application accepts student details such as name, roll number, academic percentage, CGPA, and backlog status. It processes these details using Java programming and compares them with the required placement conditions.

The main scope of the system includes:

- **Student Information:** The system accepts the necessary academic and personal details of students for eligibility checking.
- **Eligibility Checking:** It compares student details with the predefined placement criteria and determines whether the student satisfies the required conditions.
- **Result Generation:** It displays the eligibility status clearly so that students can understand whether they meet the placement requirements.
- **Data Management:** If a database is implemented, the system can store and retrieve student information for future reference.
- **Future Improvements:** The system can be enhanced to support company-wise eligibility criteria, student record management, and placement notifications.

Users and criteria. The intended users are students who wish to know whether they can apply to a company and members of the placement cell who verify eligibility for a drive; both use the same console interface. The illustrative model used throughout this report evaluates three conditions: a minimum percentage of **60**, a minimum CGPA of **6.0**, and a maximum of **0** active backlogs. These values are examples only, not official or universal requirements, and they are held in one place in the program (the PlacementCriteria class) so that an institution can change them easily.

Limitations

- The core program checks one student at a time and does not read bulk records from files.
- Only one criteria set is active in the core program; separate criteria for each company are a proposed enhancement.

- The core program does not authenticate users and does not verify the entered marks against official college records.
- Branch or course restrictions are discussed as a general criterion but are not evaluated by the core program.

Convention Used in This Report

To avoid confusion between what is built and what is only proposed, features are described using four labels: **Implemented** (part of the core console program), **Optional extension** (code and design provided but requiring database setup), **Illustrative** (example data, thresholds, durations, and screens), and **Proposed** (future enhancements). Where actual project details are not available, a basic Java-based Placement Eligibility Checker is used as the illustrative project model, and the assumptions made are stated clearly.

Ch- 2
Theoretical Background

2.1 Fundamental Concept

Campus placement is a recruitment process in which companies visit educational institutions to select students before they complete their degree. The institution's placement cell acts as the link between the companies and the students. It collects each company's requirements, announces the drive, shortlists the students who qualify, and coordinates the tests and interviews.

Eligibility checking is the first filter in this process. A company specifies the minimum standards a student must meet to be considered, and the placement cell verifies each student against those standards. A student who does not meet the standards is not allowed to participate in that drive, even if he or she is otherwise capable.

The Placement Eligibility Checker follows a **rule-based evaluation** model. In a rule-based system, decisions are produced by applying a fixed set of conditions to the input data. Each rule is a simple comparison, for example "percentage is at least the minimum percentage". The student is declared eligible only when every rule is satisfied. Because the rules are explicit, the result can always be explained by pointing to the rule that was not met.

The data used in the evaluation comes from **student academic records**: the aggregate percentage, the CGPA, and the number of subjects in which the student currently has a backlog. Software plays an important role here because it can apply the same comparison to every student without fatigue or oversight, and it can present the outcome and its explanation immediately.

2.2 Placement Eligibility Criteria

Eligibility requirements are not the same for every company and every recruitment drive. They depend on the nature of the job, the company's hiring policy, and sometimes the institution's own rules. The factors that commonly appear in such criteria are explained below.

- **Academic percentage or CGPA:** Companies often require a minimum aggregate percentage, a minimum CGPA, or both. Some also ask for minimum marks in earlier examinations.
- **Active or previous backlogs:** An active backlog is a subject that a student has not yet cleared. Many companies allow no active backlogs, while some permit a small number. Some distinguish between active backlogs and backlogs that were cleared later.
- **Degree, branch, or course:** A company may recruit only from particular branches, for example Computer Science and Information Technology, or only for a particular degree programme.
- **Company-specific criteria:** A recruiter may add conditions such as a gap-year limit, an age limit, or a minimum score in a screening test.
- **Additional criteria configured by an institution or recruiter:** An institution may add its own rules, such as minimum attendance or clearance of internal assessments.

The values used in this project are therefore **illustrative only**. The table below shows the example criteria used consistently throughout the report.

Table 2.1: Example placement criteria used in this report (illustrative, configurable)

Criterion	Rule applied in this report	Example value (illustrative)
Aggregate percentage	Student percentage must be greater than or equal to the minimum	60
CGPA	Student CGPA must be greater than or equal to the minimum	6.0
Active backlogs	Number of active backlogs must be less than or equal to the maximum	0

2.3 Benefits of the System

A software-based eligibility checker offers several practical benefits over manual checking:

- **Time saving:** A result is produced in seconds, which is valuable when hundreds of students must be checked for a drive.
- **Consistent application of rules:** The same conditions are applied to every student in exactly the same way, so two similar records always give the same result.
- **Fewer manual calculation errors:** Comparisons are made by the program, which removes mistakes caused by misreading or miscalculating marks.
- **Clear results:** The output states whether the student is eligible and lists each reason for rejection, so the student knows what to improve.
- **Easier record management:** When database support is added, student details and past results can be stored and retrieved without searching through paper files or spreadsheets.
- **Support for placement administration:** The placement cell can spend its time on coordination with companies rather than on repetitive verification.

Ch- 3
Problem Definition & System Analysis

3.1 Existing System & Problem Definition

In many colleges, eligibility checking is still done manually. The placement department receives a company's criteria, collects student marks from mark sheets, spreadsheets, or paper registers, and checks each student one by one. The list of eligible students is then prepared by hand or with the help of simple spreadsheet filters.

This existing approach has several problems:

- **Repeated calculations:** For every new company, the same students must be checked again against a new set of conditions.
- **Difficulty with large numbers of students:** As the number of students and companies grows, manual checking becomes slow and tiring.
- **Scattered records:** Marks, CGPA, and backlog details may be stored in different files or registers, which makes verification difficult.
- **Human error:** A wrongly read mark or a missed backlog can wrongly include or exclude a student.
- **Inconsistent decisions:** When different people check different batches, the same rule may be interpreted in slightly different ways.
- **Lack of explanation:** A student who is rejected often receives no clear reason.

Problem definition: There is a need for a simple, reliable tool that accepts a student's academic details, compares them with the placement criteria, and reports the eligibility result together with the reasons, without requiring manual calculation.

3.2 Proposed System

The proposed system is a Java-based console application. A user starts the program and enters the student's name, roll number, percentage, CGPA, and number of active backlogs. The program validates the entered values and rejects invalid input, such as a percentage above 100 or a negative backlog count. It then compares the validated data with the placement criteria and produces an eligibility result.

The result screen shows the student's identification, the status (ELIGIBLE or NOT ELIGIBLE), and one reason line for each condition that failed. If the student satisfies every condition, a single line states that all criteria are satisfied. The classes of the program are kept separate so that the validation, the criteria, and the checking logic can each be modified without disturbing the others. Saving the result to a relational database through JDBC is provided as an optional extension.

3.3 System Analysis

System analysis studies who will use the system, what data goes in and out, and whether the solution is practical. The analysis for this project is summarised below.

Table 3.1: Summary of system analysis

Aspect	Description
Users	Students and placement cell staff
Inputs	Student name, roll number, percentage, CGPA, number of active backlogs
Processing	Validation of inputs; comparison with minimum percentage, minimum CGPA, and maximum active backlogs; collection of reasons for failure
Outputs	Eligibility status (ELIGIBLE / NOT ELIGIBLE) and the reason(s) for the decision; optionally a stored result record
Assumptions	Entered marks are correct and supplied by an authorised person; criteria values are illustrative and set in the program; one criteria set is active at a time
Constraints	Console interface only; no user authentication; one student is processed per run; database use requires separate setup

Feasibility. The project is technically feasible because it needs only the standard Java platform for its core functionality. It is economically feasible because Java and the optional database software are available free of cost. It is operationally feasible because the program asks simple questions and presents plain-language results that need no special training. The proposed solution is suitable because the eligibility decision is a small set of well-defined comparisons, which is exactly the kind of task that a short rule-based program performs reliably.

3.4 Functional Requirements

Functional requirements describe what the system must do. They are listed in Table 3.2 together with their status in this report.

Table 3.2: Functional requirements

ID	Requirement	Description	Status
FR1	Accept student details	The system accepts name, roll number, percentage, CGPA, and active backlogs	Implemented
FR2	Validate input	The system rejects empty text, values outside valid ranges, and non-numeric input	Implemented
FR3	Define criteria	The system provides minimum percentage, minimum CGPA, and maximum active backlogs (defaults in code)	Implemented
FR4	Check eligibility	The system compares the student with every criterion	Implemented

ID	Requirement	Description	Status
FR5	Display result and reason	The system shows the status and the reason(s)	Implemented
FR6	Save and retrieve records	The system stores students and results in a relational database	Optional extension
FR7	Load criteria from storage	The system reads the criteria from the database instead of code	Proposed

3.5 Non-Functional Requirements

Non-functional requirements describe how well the system should perform its functions. They are quality attributes rather than features.

- **Usability:** Prompts are short and clear; invalid input produces an understandable message and the user is asked to try again.
- **Reliability:** Invalid data must not crash the program; exceptions are handled and reported.
- **Maintainability:** The program is divided into small classes with a single responsibility, so criteria and logic can be changed independently.
- **Performance:** A single eligibility check involves three comparisons and completes effectively instantly.
- **Data integrity:** Values are range-checked before use; the database design adds primary keys, foreign keys, and check constraints.
- **Security:** Database credentials are never written into the source code in this report (placeholders are used); user login is a proposed enhancement.
- **Portability:** Because Java programs run on the Java Virtual Machine, the same program runs on Windows, Linux, and macOS.

Table 3.3 contrasts the two kinds of requirements using examples from this project.

Table 3.3: Functional versus non-functional requirements

Aspect	Functional requirements	Non-functional requirements
Focus	What the system does	How well the system does it
Example	Check eligibility and display the reason	Respond quickly and handle bad input without crashing
Measured by	Presence and correctness of features	Quality attributes such as usability, reliability, portability
Typical items here	FR1 to FR7 (Table 3.2)	Usability, reliability, maintainability, integrity, security, portability

Ch- 4
System Planning

4.1 Introduction to System Planning

System planning is the stage in which the work of a project is thought through before the actual development begins. A good plan answers four questions: what has to be built, in what order the work will be done, how long each part is likely to take, and what resources are needed. For a small academic project such as the Placement Eligibility Checker, planning prevents the common problems of starting to code before the requirements are clear, forgetting the testing and documentation work, and running short of time near the submission date.

Planning also helps in dividing the project into manageable tasks and allocating effort sensibly among analysis, design, implementation, testing, and documentation.

4.2 Step-Wise Activities

The project is organised as the following sequence of activities:

1. **Problem identification:** Understand why manual eligibility checking is inefficient and define what the application must achieve.
2. **Requirement collection:** Identify the student details to be accepted, the criteria to be applied, and the form of the result.
3. **System analysis:** Study users, inputs, processing, outputs, constraints, and feasibility (Chapter 3).
4. **System design:** Design the classes, the data flow, the algorithm, the screens, and, if required, the database structure.
5. **Java implementation:** Write the Student, PlacementCriteria, EligibilityResult, EligibilityChecker, and Main classes.
6. **Database integration, if applicable:** Create the tables and add JDBC code for saving results. This step is optional in the illustrative model.
7. **Testing:** Check the program with valid, invalid, and boundary inputs.
8. **Documentation:** Prepare this project report with diagrams, code, and screens.
9. **Maintenance and possible enhancements:** Correct defects, update criteria values, and plan future features.

4.3 PERT Chart Flow (Sequence)

A PERT (Program Evaluation and Review Technique) chart is a network diagram in which circles represent events (milestones) and arrows represent activities that take time. It shows the order in which tasks must be completed and which tasks depend on others. Figure 4.1 presents the PERT chart for this project from requirements to final documentation.

Events are numbered circles; arrows are activities A to I; TE = cumulative (earliest) event time in days (illustrative).

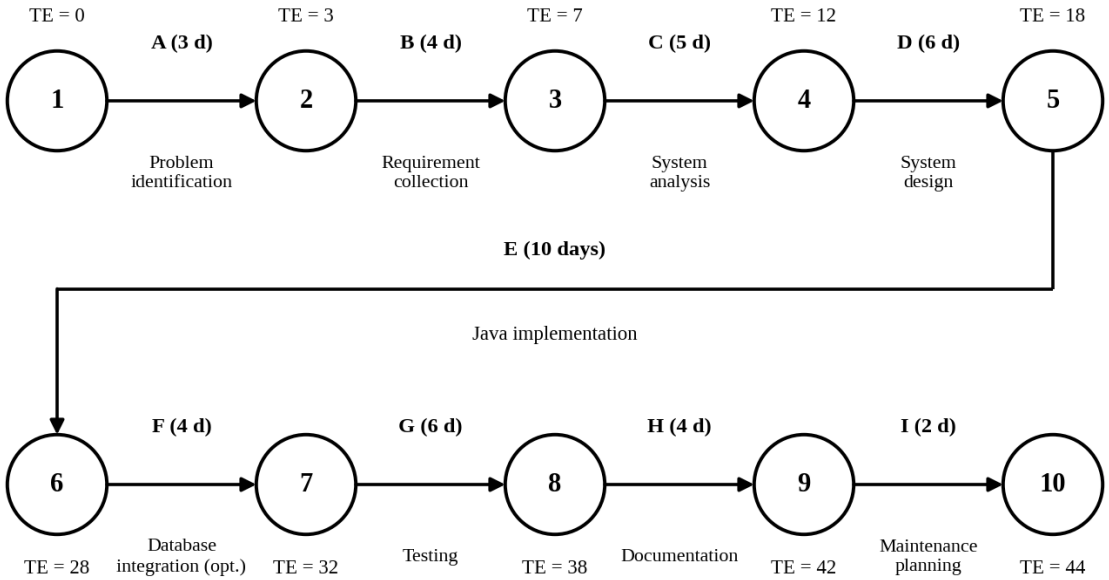


Fig. 4.1: PERT chart for the Placement Eligibility Checker (illustrative durations)

The activities, their predecessors, and their durations are listed in Table 4.1. **The durations are illustrative planning estimates; they are not the actual time spent on the project.**

Table 4.1: Activities and dependencies (illustrative)

Activity	Description	Predecessor	Duration (days)
A	Problem identification	None	3
B	Requirement collection	A	4
C	System analysis	B	5
D	System design	C	6
E	Java implementation (core)	D	10
F	Database integration (optional)	E	4
G	Testing	F	6
H	Documentation	G	4
I	Maintenance planning	H	2

In this chart every activity depends on the completion of the one before it, so the sequence forms a single path from Event 1 to Event 10, and every activity lies on the critical path. The cumulative time (TE) shown below each event adds up to 44 days. If the optional database activity (F) is skipped, the testing activity follows the core implementation directly and the illustrative total becomes 40 days. Testing cannot begin before the code exists, and documentation of the final design, code, and results can be completed only after testing, which is why the arrows are arranged in this order.

Ch- 5
Methodology Adopted and Details of Hardware and Software Used

5.1 Methodology Adopted

This project follows the **Waterfall model** of software development. The Waterfall model is a linear, sequential approach in which each phase is completed before the next one begins, and the output of one phase becomes the input of the next. The stages used are:

- **Requirement analysis:** the needs of the users and the criteria to be evaluated are gathered and written down.
- **Planning and system design:** the classes, algorithm, screens, and data structures are designed on paper.
- **Coding:** the design is converted into Java classes.
- **Testing:** the program is checked against valid, invalid, and boundary inputs.
- **Deployment and documentation:** the program is made ready to run and the report is prepared.
- **Maintenance:** defects are corrected and criteria are updated when required.

The Waterfall model is suitable for a small Java application of this kind because the requirements are few, clear, and unlikely to change during development. The project is small enough that a single developer can complete each phase in turn, and the model produces the design documents (diagrams, algorithm, and database design) that an academic report requires. Its main weakness is that late changes are costly, so a limited feedback path is shown in the life-cycle diagram of Chapter 6.

5.2 Hardware Requirements

The application is lightweight. Table 5.1 gives the **suggested minimum** hardware. These are general suggestions; the actual development machine for this project is not specified in this report.

Table 5.1: Suggested hardware requirements

Component	Suggested requirement
Processor	Dual-core processor, 2.0 GHz or higher
RAM	4 GB minimum (8 GB recommended when using an IDE and a database server together)
Available storage	At least 2 GB of free disk space for the JDK, IDE, and project files
Display	Monitor with 1366 x 768 resolution or higher
Input devices	Standard keyboard and mouse

5.3 Software Requirements

Table 5.2 lists the software needed to build and run the project. Only the JDK is essential for the core console program; the other items are options or are needed only for the database extension. No claim is made that a particular IDE, operating system, or database was actually used in developing the project.

Table 5.2: Suggested software requirements

Software	Suggested choice	Essential or optional
Java Development Kit (JDK)	JDK 17 or a later long-term-support release	Essential
IDE or editor	IntelliJ IDEA, Eclipse, or NetBeans (any one); a text editor with the command line also works	Optional
Operating system	Windows, Linux, or macOS	Essential (any one)
Relational database	MySQL 8.x (any JDBC-compatible database can be used with a suitable driver)	Optional (database extension)
JDBC driver	MySQL Connector/J matching the database	Optional (database extension)

5.4 Technologies Used

Essential technologies

- **Java:** A general-purpose, object-oriented, platform-independent programming language. Source code is compiled into bytecode that runs on the Java Virtual Machine.
- **Object-oriented programming (OOP):** The program is organised into classes. Each class groups data with the methods that work on it. Encapsulation (private fields with getter methods) protects the data, and separation of responsibilities keeps the classes small.
- **Exception handling:** Invalid data is reported by throwing an IllegalArgumentException from the Student constructor, and invalid number format is caught as a NumberFormatException in the input methods. Using try-catch blocks keeps the program from terminating abnormally.
- **Collections and input handling:** The ArrayList class stores the list of reasons for failure, and the Scanner class reads the data typed by the user in the console.

Optional technologies (database extension)

- **JDBC (Java Database Connectivity):** The standard Java API for connecting to relational databases, sending SQL statements, and reading the results.
- **Relational database concepts:** Tables, primary keys, foreign keys, and constraints are used to store student records and results in an organised and consistent way.
- **SQL:** The language used to create tables and to insert and retrieve data.

Ch- 6
Detailed Life Cycle

6.1 System Development Phases

The Software Development Life Cycle (SDLC) is the set of phases through which a software product is conceived, built, delivered, and maintained. The phases of this project are summarised in Table 6.1.

Table 6.1: Development phases

Phase	Work done in this project
Requirement analysis	Identify the student details, the three eligibility criteria, and the expected result
Planning	Prepare the activity sequence and the illustrative PERT chart (Chapter 4)
System design	Design the classes, DFDs, ER diagram, architecture, algorithm, flowchart, and screens (Chapters 7 to 9)
Coding	Write the Java classes described in Chapter 10
Testing	Check the code with valid, invalid, and boundary inputs (Section 6.3)
Deployment	Compile the classes and run the program on a computer that has the JDK; set up the database only if the extension is used
Documentation	Prepare the project report with diagrams and code
Maintenance	Correct defects and change criteria values or add features when required

6.2 System Design and Implementation

The life cycle followed by the project is shown in Figure 6.1. Each phase flows into the next, and a dashed feedback path indicates that defects found during maintenance or change requests can send the work back to the requirement stage.

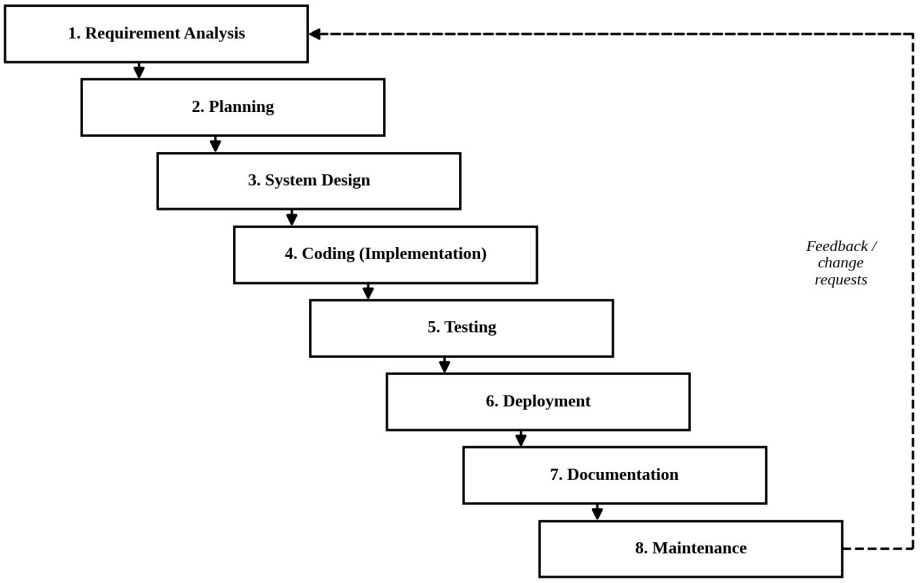


Fig. 6.1: Software Development Life Cycle (Waterfall) of the project

The design of the application is based on five Java classes, each with a specific responsibility:

- **Student** holds the entered details and validates them when an object is created.
- **PlacementCriteria** holds the minimum percentage, minimum CGPA, and maximum active backlogs, and supplies the default illustrative values.
- **EligibilityChecker** contains the decision logic through the method checkEligibility(Student, PlacementCriteria).
- **EligibilityResult** carries the final status and the list of reasons back to the caller.
- **Main** reads the input, calls the checker, and prints the result.

The **data flow** inside the program is straightforward. Main reads the typed values and creates a Student object, which validates them. Main obtains the criteria from PlacementCriteria.defaultCriteria() and passes both objects to EligibilityChecker. The checker compares the values, builds the list of reasons, and returns an EligibilityResult. Main then prints the status and reasons. In the optional extension, the ResultDao class receives the same EligibilityResult object and stores it through JDBC.

6.3 Testing, Development and Maintenance

Testing is carried out to confirm that the program behaves as intended. The following kinds of testing are relevant to this project:

- **Unit testing:** Each class is checked separately, for example the Student constructor with invalid values and EligibilityChecker with known student and criteria objects.
- **Input validation testing:** Empty names, percentages outside 0 to 100, CGPA outside 0 to 10, negative backlogs, and non-numeric text are entered to confirm that errors are reported and the user is prompted again.

- **Integration testing, where applicable:** When the database extension is used, the saving of a result is tested after the schema is created, to confirm that the Java code and the table definitions agree.
- **Error handling:** The program must display a message instead of terminating when invalid data is entered.
- **Defect correction:** Any incorrect output is traced to the responsible class, corrected, and the tests are repeated.

Table 6.2 gives **proposed example test cases** with their expected outcomes. They describe what should happen according to the program logic; no actual test execution results are claimed in this report.

Table 6.2: Proposed example test cases

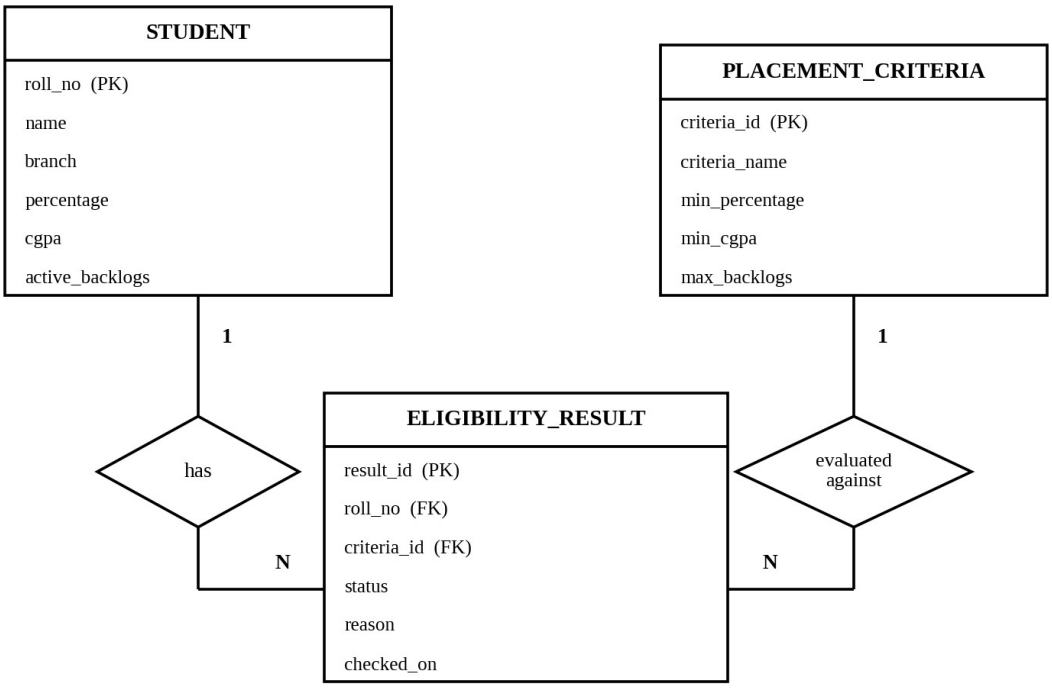
No.	Input (percentage, CGPA, backlogs)	Expected outcome
T1	72.5, 7.4, 0 (Rohan Verma, EX101)	ELIGIBLE: all criteria satisfied
T2	58, 6.5, 1 (Asha Rao, EX102)	NOT ELIGIBLE: percentage below minimum; backlogs above allowed
T3	60, 6.0, 0 (boundary values)	ELIGIBLE: values equal to the limits are accepted
T4	105, 7.0, 0	Input error: percentage must be between 0 and 100; details are asked again
T5	"abc" typed for CGPA	Message "Invalid number"; CGPA is asked again
T6	Empty student name	Input error: name must not be empty; details are asked again

Maintenance involves updating criteria values when a recruiter changes its requirements, fixing defects discovered during use, and extending the program with new features. Because the criteria are isolated in the PlacementCriteria class, a change in a limit requires editing only one place in the code.

Ch- 7
ERD & DFD

7.1 Entity Relationship Diagram

An Entity Relationship Diagram (ERD) describes the data to be stored and the relationships between the data items. It is the basis for the database design in Section 8.3. The diagram in Figure 7.1 applies when the optional database extension is used.



PK = primary key, FK = foreign key. 1 : N = one-to-many. A student can have many results; a criteria set is used in many results.

Fig. 7.1: Entity Relationship Diagram of the Placement Eligibility Checker

The diagram contains three entities:

- **STUDENT** stores the details of a student. Its primary key is roll_no. The attributes name, percentage, cgpa, and active_backlogs are used in the eligibility decision, and branch is an optional descriptive attribute that the core console program does not read.
- **PLACEMENT_CRITERIA** stores a named set of rules. Its primary key is criteria_id, and the attributes min_percentage, min_cgpa, and max_backlogs hold the limits.
- **ELIGIBILITY_RESULT** stores the outcome of one check. Its primary key is result_id. The attribute roll_no is a foreign key referring to STUDENT, and criteria_id is a foreign key referring to PLACEMENT_CRITERIA. The attributes status, reason, and checked_on record the decision, its explanation, and the time of the check.

The relationships are both **one-to-many (1 : N)**. One student can have many eligibility results, because the same student may be checked for many drives or checked again after the marks are updated. One criteria set can be used in many results, because it is applied to many students. The result entity therefore sits between the other two and carries both foreign keys.

7.2 Data Flow Diagram

A Data Flow Diagram (DFD) shows how data moves through a system. Rectangles are external entities (sources and destinations of data), ellipses are processes, and open-ended rectangles are data stores. Arrows show the direction and name of each data flow.

Context-Level DFD (Level 0)

The context diagram in Figure 7.2 shows the whole system as one process. The Student or User supplies the student details and receives the eligibility result with its reason. The Placement Officer supplies the placement criteria that the system applies.

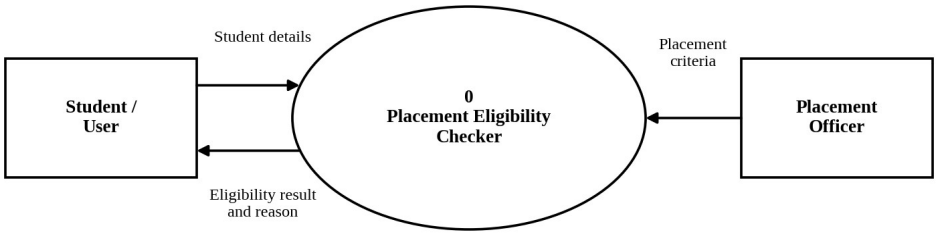


Fig. 7.2: Context-level DFD (Level 0)

Level-1 DFD

Figure 7.3 expands the single process into four sub-processes. Process 1.0 accepts and validates the student details. Process 2.0 obtains the applicable criteria from the data store D1. Process 3.0 evaluates eligibility by comparing the validated data with the criteria. Process 4.0 displays the result to the user. Data store D1 holds the placement criteria, which the Placement Officer supplies. Data store D2 holds saved records and is shown with a dashed flow because saving is an optional extension.

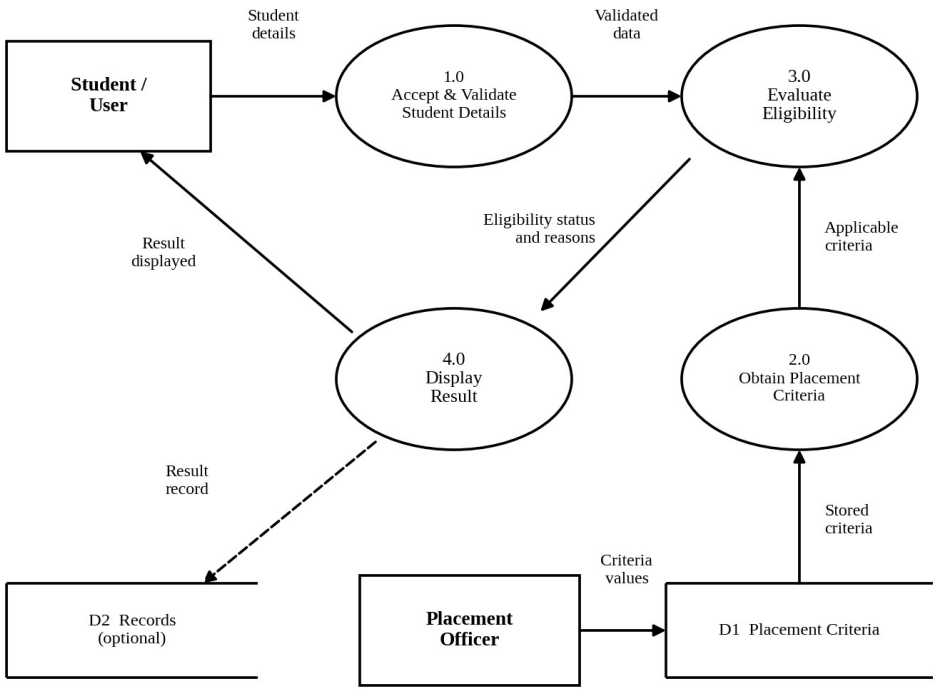


Fig. 7.3: Level-1 DFD

In the core console program, D1 corresponds to the default values defined in the PlacementCriteria class. In the database extension, the same data would be kept in the PlacementCriteria table.

7.3 System Architecture Diagram

The architecture of the application is arranged in layers, as shown in Figure 7.4. The user interacts with a console interface (the Main class). The Main class calls the application logic layer, which contains Student, PlacementCriteria, EligibilityChecker, and EligibilityResult. These layers form the core implementation and are drawn with solid lines. The data access layer (ResultDao and DatabaseConnection), the JDBC API with the database driver, and the relational database are drawn with dashed lines because they are an optional extension. The interface is console-based; a graphical interface is not part of the implemented design.

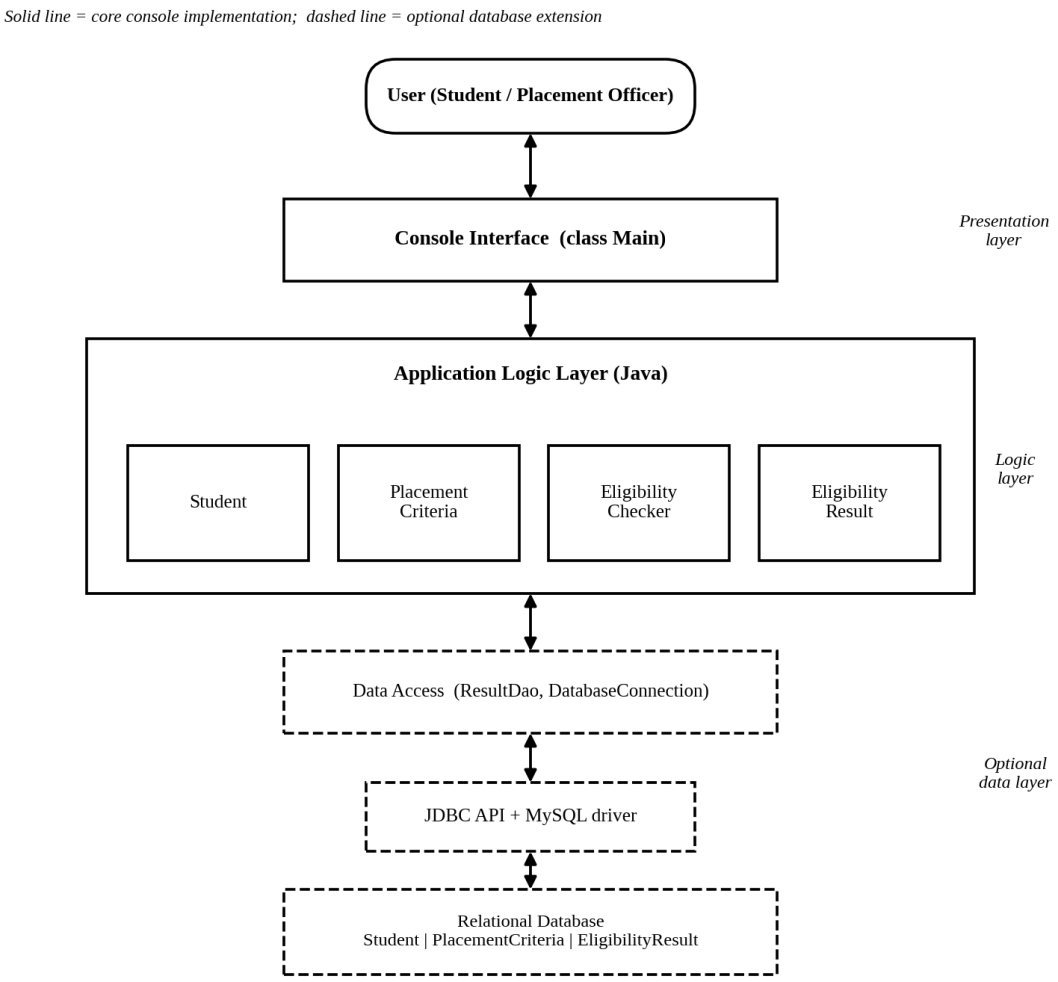


Fig. 7.4: System architecture of the Placement Eligibility Checker

The architecture is consistent with the DFDs: the processes 1.0 to 4.0 of the Level-1 diagram are carried out by the console interface and the logic layer, while the data stores D1 and D2 correspond to the criteria values and, optionally, the database.

Ch- 8
Process, Algorithm, and Database Structure

8.1 System Process

The complete workflow of the application is as follows:

1. **Start the application.** The program prints its title and loads the default criteria (percentage 60, CGPA 6.0, maximum active backlogs 0; illustrative).
2. **Enter student details.** The user types name, roll number, percentage, CGPA, and number of active backlogs.
3. **Validate input.** Non-numeric values are rejected by the input methods, and out-of-range or empty values are rejected by the Student constructor. On error, the user is asked to re-enter the details.
4. **Obtain placement criteria.** The criteria object supplies the limits.
5. **Compare the details with each condition.** Percentage, CGPA, and active backlogs are checked one after another, and a reason is recorded for each failed condition.
6. **Determine eligibility.** The student is eligible only if no reason was recorded.
7. **Display the result and the relevant reason.** The status and every reason are printed.
8. **Save the result, if persistent storage is implemented.** In the optional extension, ResultDao stores the student and the result.
9. **End the process.**

8.2 Algorithm

The numbered algorithm below performs the eligibility check. It corresponds to the flowchart in Figure 8.1 and to the method checkEligibility in Chapter 10.

1. Start.
2. Read name, roll number, percentage, cgpa, and activeBacklogs.
3. If any value is invalid (empty text, percentage outside 0 to 100, cgpa outside 0 to 10, or negative backlogs), display an error message and go to Step 2.
4. Load the criteria: minPercentage = 60, minCgpa = 6.0, maxBacklogs = 0 (illustrative defaults).
5. Create an empty list called reasons.
6. If percentage < minPercentage, add the percentage reason to reasons.
7. If cgpa < minCgpa, add the CGPA reason to reasons.
8. If activeBacklogs > maxBacklogs, add the backlog reason to reasons.
9. If reasons is empty, set status = ELIGIBLE and add the message that all criteria are satisfied; otherwise set status = NOT ELIGIBLE.
10. Display the student identification, the criteria, the status, and every reason.

11. Optionally, save the result to the database.
12. Stop.

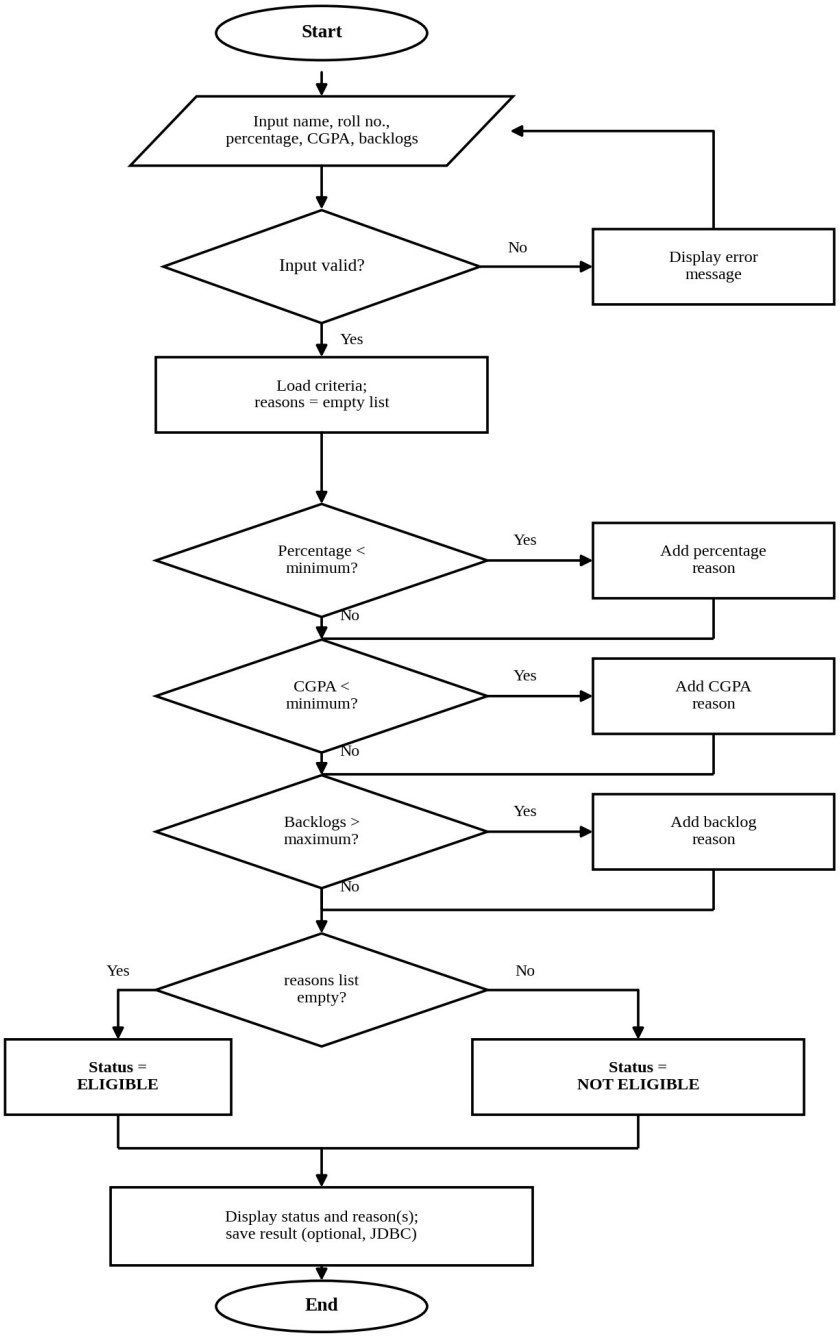


Fig. 8.1: Flowchart of the eligibility checking process

The flowchart shows that the three comparisons are executed one after another and that a failed condition only adds a reason; it does not stop the process. This design lets the user see **all** the reasons for rejection at once instead of only the first one. Boundary values are accepted, because the conditions test "less than minimum" and "greater than maximum".

8.3 Database Design

The core console program does not need a database, so database integration is an **optional extension**. When it is used, the design follows the ER diagram of Chapter 7 and consists of three relational tables. Tables 8.1 to 8.3 list the fields, data types, keys, purposes, and constraints.

Table 8.1: Structure of the Student table

Field	Data type	Key	Purpose and constraints
roll_no	VARCHAR(20)	PK	Unique roll number of the student
name	VARCHAR(100)		Student name; NOT NULL
branch	VARCHAR(50)		Optional branch; not captured by the console program
percentage	DECIMAL(5,2)		Aggregate percentage; NOT NULL; CHECK 0 to 100
cgpa	DECIMAL(4,2)		CGPA; NOT NULL; CHECK 0 to 10
active_backlogs	INT		Number of active backlogs; NOT NULL; DEFAULT 0; CHECK >= 0

Table 8.2: Structure of the PlacementCriteria table

Field	Data type	Key	Purpose and constraints
criteria_id	INT	PK	Auto-incremented identifier of the criteria set
criteria_name	VARCHAR(100)		Name of the criteria set; NOT NULL
min_percentage	DECIMAL(5,2)		Minimum percentage; NOT NULL
min_cgpa	DECIMAL(4,2)		Minimum CGPA; NOT NULL
max_backlogs	INT		Maximum active backlogs allowed; NOT NULL

Table 8.3: Structure of the EligibilityResult table

Field	Data type	Key	Purpose and constraints
result_id	INT	PK	Auto-incremented identifier of the result
roll_no	VARCHAR(20)	FK	References Student(roll_no); NOT NULL
criteria_id	INT	FK	References PlacementCriteria(criteria_id); NOT NULL
status	VARCHAR(15)		ELIGIBLE or NOT ELIGIBLE; NOT NULL; CHECK constraint
reason	VARCHAR(500)		Reason(s) for the decision, joined by semicolons
checked_on	TIMESTAMP		Time of the check; DEFAULT CURRENT_TIMESTAMP

Relationships and integrity. The foreign keys roll_no and criteria_id ensure that a result cannot refer to a student or criteria set that does not exist. Primary keys prevent duplicate records, and the CHECK and NOT NULL constraints keep invalid values out of the tables. Record management is done by inserting a student record (or updating it if the roll number already exists), inserting a result row for every check, and retrieving past results by roll number. The following SQL statements create the tables and one default criteria row; they were written for MySQL 8.0.16 or later, because earlier versions do not enforce CHECK constraints.

```
CREATE TABLE Student (
  roll_no VARCHAR(20) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  branch VARCHAR(50),
  percentage DECIMAL(5,2) NOT NULL CHECK (percentage BETWEEN 0 AND 100),
  cgpa DECIMAL(4,2) NOT NULL CHECK (cgpa BETWEEN 0 AND 10),
  active_backlogs INT NOT NULL DEFAULT 0 CHECK (active_backlogs >= 0)
);

CREATE TABLE PlacementCriteria (
  criteria_id INT AUTO_INCREMENT PRIMARY KEY,
  criteria_name VARCHAR(100) NOT NULL,
  min_percentage DECIMAL(5,2) NOT NULL,
  min_cgpa DECIMAL(4,2) NOT NULL,
  max_backlogs INT NOT NULL
);

CREATE TABLE EligibilityResult (
  result_id INT AUTO_INCREMENT PRIMARY KEY,
  roll_no VARCHAR(20) NOT NULL,
  criteria_id INT NOT NULL,
  status VARCHAR(15) NOT NULL CHECK (status IN ('ELIGIBLE','NOT ELIGIBLE')),
  reason VARCHAR(500),
  checked_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (roll_no) REFERENCES Student(roll_no),
  FOREIGN KEY (criteria_id) REFERENCES PlacementCriteria(criteria_id)
);

INSERT INTO PlacementCriteria (criteria_name, min_percentage, min_cgpa, max_backlogs)
VALUES ('Default (illustrative)', 60.00, 6.00, 0);
```

Ch- 9
Input and Output Screen

9.1 User Interface Overview

The application uses a **console (text-based) interface**. The user runs the program in a terminal or in the console window of an IDE. The program displays a title and then prompts for each value on its own line. After the last value is entered, the program processes the data and prints the result in the same window. Information therefore moves in three steps: the user types the input, the Student and EligibilityChecker classes process it, and Main prints the output.

The screens below are **illustrative mock-ups of console output**. They were prepared using the sample data of this report and are not screenshots of a program run on the college's systems. The sample students are fictitious.

9.2 Student Details Input Screen

The input screen asks for the student name, roll number, percentage, CGPA, and number of active backlogs. Values typed by the user appear after each colon. Figure 9.1 shows the entry of the first sample student.

```
===== PLACEMENT ELIGIBILITY CHECKER =====
Enter Student Name   : Rohan Verma
Enter Roll Number    : EX101
Enter Percentage     : 72.5
Enter CGPA           : 7.4
Enter Active Backlogs: 0
```

Fig. 9.1: Student details input screen (illustrative console mock-up)

If a non-numeric value is typed for a number field, the program prints a message such as "Invalid number. Please try again." and asks for the same field again. If a value is outside its allowed range or a text field is empty, the program prints an input error and asks for all details again.

9.3 Eligibility Result Screen

The result screen shows the student identification, the criteria evaluated, the eligibility status, and the reason(s). Figure 9.2 shows the result for two sample students: part (a) for an eligible student and part (b) for a student who is not eligible.

```
(a)  Rohan Verma  (72.5, 7.4, 0 backlogs)

===== ELIGIBILITY RESULT =====
Name           : Rohan Verma
Roll Number    : EX101
Criteria       : Percentage >= 60.0, CGPA >= 6.0, Active backlogs <= 0
Status         : ELIGIBLE
Reason         : All placement criteria are satisfied.
```

Fig. 9.2(a): Result screen for an eligible student (illustrative)

```
(b)  Asha Rao  (58, 6.5, 1 backlog)

===== ELIGIBILITY RESULT =====
Name           : Asha Rao
Roll Number    : EX102
Criteria       : Percentage >= 60.0, CGPA >= 6.0, Active backlogs <= 0
Status         : NOT ELIGIBLE
Reason         : Percentage 58.0 is below the minimum 60.0
Reason         : Active backlogs 1 exceed the allowed 0
```

Fig. 9.2(b): Result screen for a student who is not eligible (illustrative)

In part (b), the student satisfies the CGPA condition but fails the percentage and backlog conditions; the screen therefore lists two reasons. This tells the student exactly which requirements were not met. The screens are consistent with the Main class and the EligibilityChecker class shown in Chapter 10.

Ch- 10
Coding (Java Implementation)

10.1 Java Program Structure

The program consists of five core classes, which make up the console implementation, and two optional classes for database connectivity. All classes are in the default package and can be compiled together. Table 10.1 describes their roles and status.

Table 10.1: Classes of the Placement Eligibility Checker

Class	Responsibility	Status
Student	Stores name, roll number, percentage, CGPA, and active backlogs; validates the values in its constructor	Implemented (core)
PlacementCriteria	Stores minimum percentage, minimum CGPA, and maximum active backlogs; provides illustrative defaults	Implemented (core)
EligibilityResult	Holds the student, the eligibility flag, and the list of reasons	Implemented (core)
EligibilityChecker	Compares a Student with a PlacementCriteria and produces an EligibilityResult	Implemented (core)
Main	Reads input, calls the checker, and prints the result	Implemented (core)
DatabaseConnection	Creates a JDBC connection using placeholder settings	Optional extension
ResultDao	Saves the student and the result using prepared statements	Optional extension

The core code uses only classes from the standard Java library (java.util.Scanner, java.util.List, and java.util.ArrayList), so it needs no external library. The sample thresholds are illustrative and configurable by changing the values in PlacementCriteria.defaultCriteria().

10.2 Eligibility Checking Module

The listings below give the complete core program. **Student.java** keeps the entered information and performs the input validation. It throws an IllegalArgumentException with a clear message when a value is invalid.

```
public class Student {
    private final String name;
    private final String rollNumber;
    private final double percentage;
    private final double cgpa;
    private final int activeBacklogs;

    public Student(String name, String rollNumber, double percentage,
                   double cgpa, int activeBacklogs) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Student name must not be empty.");
        }
        if (rollNumber == null || rollNumber.trim().isEmpty()) {
            throw new IllegalArgumentException("Roll number must not be empty.");
        }
        if (percentage < 0 || percentage > 100) {
            throw new IllegalArgumentException("Percentage must be between 0 and 100.");
        }
        if (cgpa < 0 || cgpa > 10) {
            throw new IllegalArgumentException("CGPA must be between 0 and 10.");
```

```
        }
        if (activeBacklogs < 0) {
            throw new IllegalArgumentException("Backlogs cannot be negative.");
        }
        this.name = name.trim();
        this.rollNumber = rollNumber.trim();
        this.percentage = percentage;
        this.cgpa = cgpa;
        this.activeBacklogs = activeBacklogs;
    }

    public String getName()          { return name; }
    public String getRollNumber()    { return rollNumber; }
    public double getPercentage()    { return percentage; }
    public double getCgpa()          { return cgpa; }
    public int getActiveBacklogs()   { return activeBacklogs; }
}
```

PlacementCriteria.java stores the eligibility criteria. The static method defaultCriteria() returns the illustrative values 60, 6.0, and 0.

```
public class PlacementCriteria {
    private final double minPercentage;
    private final double minCgpa;
    private final int maxActiveBacklogs;

    public PlacementCriteria(double minPercentage, double minCgpa,
                             int maxActiveBacklogs) {
        this.minPercentage = minPercentage;
        this.minCgpa = minCgpa;
        this.maxActiveBacklogs = maxActiveBacklogs;
    }

    public static PlacementCriteria defaultCriteria() {
        return new PlacementCriteria(60.0, 6.0, 0);
    }

    public double getMinPercentage()   { return minPercentage; }
    public double getMinCgpa()         { return minCgpa; }
    public int getMaxActiveBacklogs()  { return maxActiveBacklogs; }
}
```

EligibilityResult.java carries the outcome of the check from the checker back to the caller.

```
import java.util.ArrayList;
import java.util.List;

public class EligibilityResult {
    private final Student student;
    private final boolean eligible;
    private final List<String> reasons;

    public EligibilityResult(Student student, boolean eligible,
                             List<String> reasons) {
        this.student = student;
        this.eligible = eligible;
        this.reasons = new ArrayList<>(reasons);
    }

    public Student getStudent()       { return student; }
    public boolean isEligible()       { return eligible; }
    public List<String> getReasons()  { return reasons; }
    public String getStatus()         { return eligible ? "ELIGIBLE" : "NOT ELIGIBLE"; }
}
```

EligibilityChecker.java contains the decision logic. The method checkEligibility() performs the percentage check, the CGPA check, and the backlog check. Each failed comparison appends a descriptive reason to a list. When the list is empty after all three checks, the student is eligible and a confirming message is added. This is exactly the algorithm of Section 8.2.

```
import java.util.ArrayList;
import java.util.List;
```

```
public class EligibilityChecker {

    public EligibilityResult checkEligibility(Student student,
                                               PlacementCriteria criteria) {
        List<String> reasons = new ArrayList<>();

        if (student.getPercentage() < criteria.getMinPercentage()) {
            reasons.add("Percentage " + student.getPercentage()
                    + " is below the minimum " + criteria.getMinPercentage());
        }
        if (student.getCgpa() < criteria.getMinCgpa()) {
            reasons.add("CGPA " + student.getCgpa()
                    + " is below the minimum " + criteria.getMinCgpa());
        }
        if (student.getActiveBacklogs() > criteria.getMaxActiveBacklogs()) {
            reasons.add("Active backlogs " + student.getActiveBacklogs()
                    + " exceed the allowed " + criteria.getMaxActiveBacklogs());
        }

        boolean eligible = reasons.isEmpty();
        if (eligible) {
            reasons.add("All placement criteria are satisfied.");
        }
        return new EligibilityResult(student, eligible, reasons);
    }
}
```

Main.java is the entry point. The methods readDouble() and readInt() repeat the prompt until a valid number is typed, using try-catch to handle NumberFormatException. The Student constructor may throw an IllegalArgumentException; Main catches it, prints the message, and asks for the details again. After a valid Student is created, the checker is called and the result is printed.

```
import java.util.Scanner;

public class Main {
    private static final Scanner SC = new Scanner(System.in);

    private static double readDouble(String prompt) {
        while (true) {
            System.out.print(prompt);
            try {
                return Double.parseDouble(SC.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Invalid number. Please try again.");
            }
        }
    }

    private static int readInt(String prompt) {
        while (true) {
            System.out.print(prompt);
            try {
                return Integer.parseInt(SC.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Invalid whole number. Please try again.");
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("===== PLACEMENT ELIGIBILITY CHECKER =====");
        PlacementCriteria criteria = PlacementCriteria.defaultCriteria();
        EligibilityChecker checker = new EligibilityChecker();

        Student student = null;
        while (student == null) {
            System.out.print("Enter Student Name   : ");
            String name = SC.nextLine();
            System.out.print("Enter Roll Number    : ");
            String roll = SC.nextLine();
            double percentage = readDouble("Enter Percentage     : ");
            double cgpa = readDouble("Enter CGPA           : ");
            int backlogs = readInt("Enter Active Backlogs: ");
            try {
                student = new Student(name, roll, percentage, cgpa, backlogs);
            } catch (IllegalArgumentException e) {
                System.out.println("Input error: " + e.getMessage());
                System.out.println("Please re-enter the details.\n");
            }
```

```
        }

        EligibilityResult result = checker.checkEligibility(student, criteria);

        System.out.println("\n===== ELIGIBILITY RESULT =====");
        System.out.println("Name           : " + student.getName());
        System.out.println("Roll Number    : " + student.getRollNumber());
        System.out.println("Criteria       : Percentage >= " + criteria.getMinPercentage()
                + ", CGPA >= " + criteria.getMinCgpa()
                + ", Active backlogs <= " + criteria.getMaxActiveBacklogs());
        System.out.println("Status         : " + result.getStatus());
        for (String reason : result.getReasons()) {
            System.out.println("Reason         : " + reason);
        }
    }
}
```

To run the core program, compile all .java files with the javac command and start the program with "java Main". With the input of Figure 9.1, the program prints the output shown in Figure 9.2(a).

10.3 Database Connectivity Using JDBC

JDBC (Java Database Connectivity) is the standard Java API for working with relational databases. A **database driver** is a library supplied by the database vendor (for MySQL, MySQL Connector/J) that implements the JDBC interfaces. A **Connection** object represents a session with the database, and it is obtained by calling DriverManager.getConnection() with a URL, a user name, and a password. A **PreparedStatement** holds an SQL statement with ? placeholders, and values are bound with methods such as setString() and setDouble(). Prepared statements are preferred over joining strings because they prevent SQL injection and handle data types correctly. Database resources must be closed after use, and the try-with-resources statement closes them automatically. Database errors are signalled by SQLException, which must be caught.

This section is an optional extension. The core program of Section 10.2 does not use it. Before the JDBC code can run, the following must be prepared: the database must be installed and running, the schema of Section 8.3 must be created (including the default criteria row, whose criteria_id is 1), the JDBC driver JAR must be on the classpath, and the placeholder values in DatabaseConnection must be replaced with real settings. No real credentials appear in this report. The JDBC code is based on the standard java.sql API; the ON DUPLICATE KEY UPDATE clause is MySQL syntax. In this report the JDBC classes are presented as a design and code example, and no execution against a database is claimed.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    // Placeholders: replace with your own configuration. Never commit real credentials.
    private static final String URL      = "jdbc:mysql://localhost:3306/<DATABASE_NAME>";
    private static final String USER     = "<DB_USERNAME>";
    private static final String PASSWORD = "<DB_PASSWORD>";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

The class ResultDao saves the student (inserting it, or updating it if the roll number already exists) and then inserts one row into EligibilityResult. To use it, Main would call "new ResultDao().save(result, 1);" after displaying the result. The field names in the SQL statements agree with the tables of Section 8.3.

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ResultDao {

    private static final String INSERT_STUDENT =
        "INSERT INTO Student (roll_no, name, percentage, cgpa, active_backlogs) "
      + "VALUES (?, ?, ?, ?, ?) "
      + "ON DUPLICATE KEY UPDATE name=VALUES(name), percentage=VALUES(percentage), "
      + "cgpa=VALUES(cgpa), active_backlogs=VALUES(active_backlogs)";

    private static final String INSERT_RESULT =
        "INSERT INTO EligibilityResult (roll_no, criteria_id, status, reason) "
      + "VALUES (?, ?, ?, ?)";

    public void save(EligibilityResult result, int criteriaId) {
        Student s = result.getStudent();
        try (Connection con = DatabaseConnection.getConnection();
             PreparedStatement ps1 = con.prepareStatement(INSERT_STUDENT);
             PreparedStatement ps2 = con.prepareStatement(INSERT_RESULT)) {

            ps1.setString(1, s.getRollNumber());
            ps1.setString(2, s.getName());
            ps1.setDouble(3, s.getPercentage());
            ps1.setDouble(4, s.getCgpa());
            ps1.setInt(5, s.getActiveBacklogs());
            ps1.executeUpdate();

            ps2.setString(1, s.getRollNumber());
            ps2.setInt(2, criteriaId);
            ps2.setString(3, result.getStatus());
            ps2.setString(4, String.join("; ", result.getReasons()));
            ps2.executeUpdate();
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Ch- 11
Conclusion

The purpose of the Placement Eligibility Checker is to make the first step of campus recruitment, the verification of eligibility, quick, accurate, and consistent. In many institutions this task is still performed manually, which consumes time, invites calculation mistakes, and can lead to inconsistent decisions. This project addresses that problem with a small Java application that evaluates a student against defined placement criteria and explains the outcome.

The solution accepts the student's name, roll number, percentage, CGPA, and number of active backlogs. It validates the data, compares it with the minimum percentage, minimum CGPA, and maximum active backlogs, and displays the result together with a reason for every failed condition. The program is built from separate classes (Student, PlacementCriteria, EligibilityChecker, EligibilityResult, and Main), which keeps the code readable and easy to modify. An optional JDBC extension shows how results can be stored in a relational database.

Advantages. The system saves time, applies the same rules to every student, reduces manual calculation errors, and gives clear explanations. Its criteria can be changed in one place. **Limitations.** The core program is console-based, evaluates one student at a time, uses a single criteria set, performs no user authentication, and relies on the correctness of the data typed by the user. The thresholds used in this report are illustrative, and database storage is an optional extension that requires separate setup. No quantitative improvement, deployment result, or user feedback is claimed.

Knowledge gained. While preparing the project, the following concepts were studied and applied: object-oriented design with encapsulation, input validation and exception handling in Java, rule-based decision logic, use of collections, the Waterfall development process, the preparation of ER diagrams, DFDs, flowcharts, and architecture diagrams, relational database design with keys and constraints, and the basics of JDBC with prepared statements.

Ch- 12
Future Enhancement

The Placement Eligibility Checker can be extended in many ways. The enhancements below are **proposed possibilities**; they are not implemented in the illustrative design described in this report. Table 12.1 separates what is present from what is proposed.

- **Company-wise eligibility criteria:** Store separate criteria for each company and check a student against any selected company, including branch and course conditions.
- **Graphical user interface:** Replace the console with a form-based interface (for example, using Java Swing or JavaFX) that is easier for students to use.
- **Database-backed student records:** Maintain a permanent student database, load criteria from it, and allow records to be updated and searched.
- **Secure student and administrator login:** Add authentication so that students can view only their own results and only administrators can change criteria.
- **Automated eligibility reports:** Check all students in a batch at once and produce a printable list of eligible students for each drive.
- **Placement notifications:** Send an email or message to eligible students when a new drive is announced.
- **Better validation:** Verify entered marks against official college records and detect duplicate or inconsistent entries.
- **Integration with a college placement portal:** Connect the checker to the college's placement web portal so that eligibility is determined automatically when students register for a drive.

Table 12.1: Implemented, optional, and proposed features

Feature	Status in this report
Console input and validation	Implemented (core)
Percentage, CGPA, and backlog checking with reasons	Implemented (core)
Default criteria held in code	Implemented (core)
Saving results through JDBC	Optional extension (code and design provided)
Company-wise criteria, GUI, login, reports, notifications, portal integration	Proposed future enhancement

References

[1] Oracle Corporation. The Java Tutorials. https://docs.oracle.com/javase/tutorial/

[2] Oracle Corporation. Java Platform, Standard Edition 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/

[3] Oracle Corporation. The Java Tutorials: JDBC Database Access (JDBC Basics). https://docs.oracle.com/javase/tutorial/jdbc/basics/

[4] Oracle Corporation. The Java Tutorials: Lesson: Exceptions. https://docs.oracle.com/javase/tutorial/essential/exceptions/

[5] Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[6] Oracle Corporation. MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

[7] Schildt, H. Java: The Complete Reference. McGraw-Hill (Oracle Press).

[8] Bloch, J. Effective Java, 3rd Edition. Addison-Wesley, 2018.

[9] Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts. McGraw-Hill Education.

[10] Elmasri, R., and Navathe, S. B. Fundamentals of Database Systems. Pearson.

[11] Sommerville, I. Software Engineering, 10th Edition. Pearson, 2015.

[12] Pressman, R. S., and Maxim, B. R. Software Engineering: A Practitioner's Approach, 9th Edition. McGraw-Hill Education, 2019.